

An improved algorithm for the red-blue hitting set problem with the consecutive ones property

Maw-Shang Chang*, Hsiao-Han Chung^{†‡}, Chuang-Chieh Lin[§]

*Department of Computer Science and Information Engineering,
National Chung Cheng University,
Ming-Hsiung, Chiayi, Taiwan.*

Abstract

Covering problems are important in many applications. In this work, we consider the red-blue hitting set problem, which is a special covering problem. Assume that we are given a set of elements S and two collections $\mathcal{C}_r$ and $\mathcal{C}_b$ of subsets of S , which are called a red collection and a blue collection respectively. We say that a set $A \in \mathcal{C}_r \cup \mathcal{C}_b$ is hit by a subset $S' \subseteq S$ if $A \cap S' \neq \emptyset$. The red-blue hitting set problem asks for a subset $S' \subseteq S$ such that all sets in the blue collection must be hit by S' , while the number of sets in the red collection hit by S' has to be minimum. This problem is polynomially solvable when the sets in the blue collection and the red collection have the consecutive ones property. We present a shortest-path-based algorithm for this problem with the consecutive ones property and its time complexity is $O(|\mathcal{C}_b||S| + |\mathcal{C}_r||S| + |S|^2)$, which improves the previous time bound $O(|\mathcal{C}_r||\mathcal{C}_b||S|^2)$ by Dom et al. [6].

Keywords: Covering problem; Hitting set; Consecutive ones property; Shortest path.

1 Introduction

Covering problems have played a central role in algorithmics and combinatorial optimization. The *set cover problem* and the *hitting set problem* are two of the best-known covering problems. Assume that we have a set S and a collection $\mathcal{C}$ of subsets of S as the input. We say that a subset $X \subseteq S$ is *hit by an element* $s \in S$ (or s is covered by X) if

$s \in X$. Generally, a set X is said to be *hit by a subset* $S' \subseteq S$ if $X \cap S' \neq \emptyset$ (or $S' \cap X$ is covered by X). The set cover problem is to find a minimum size subcollection $\mathcal{C}' \subseteq \mathcal{C}$ such that $\bigcup_{C \in \mathcal{C}'} C = S$ (i.e., S is *covered* by $\mathcal{C}$). By exchanging elements and sets, the set cover problem is equivalent to the hitting set problem, which is to find a minimum size subset $S' \subseteq S$ such that each set in $\mathcal{C}$ contains at least one element from S' (i.e., $\mathcal{C}$ is hit by S'). The set cover problem is well-known to be NP-hard and only allows for a logarithmic-factor polynomial-time approximation [7].

In this paper, we study a covering problem, called the *red-blue hitting set problem* (RBHS), which is the hitting set problem with the “red-blue” restriction.

The red-blue hitting set problem (RBHS).

Input: A set S , two collections $\mathcal{C}_b$ (blue collection) and $\mathcal{C}_r$ (red collection) of subsets of S .

Task: Find a subset $S' \subseteq S$ such that for each $X \in \mathcal{C}_b$, $|S' \cap X| \geq 1$ and $|\{X \in \mathcal{C}_r : S' \cap X \neq \emptyset\}|$ is minimized.

The above definition is given in [6]. Carr et al. [2] discussed the *red-blue set cover problem*, which is equivalent to RBHS by simply exchanging sets and elements. And the *red-blue set cover problem* is discussed in [5, 9].

The application scenarios of RBHS range over data mining to interference reduction in cellular networks and computational biology [1]. A further application problem connected to RBHS is the *generalized venetian routing problem*, which deals with wavelength routing in optical networks [3].

By letting $\mathcal{C}_r = \mathcal{C}_b$ we can derive that the original hitting set problem is a special case of RBHS. Thus, RBHS is computationally at least as hard as the hitting set problem. Here let us consider a 0-1 matrix M , where elements of S correspond

*E-Mail: mschang@cs.ccu.edu.tw

†E-Mail: zzh96m@cs.ccu.edu.tw

‡Corresponding author.

§E-Mail: lincc@cs.ccu.edu.tw

to columns of M and sets in $\mathcal{C}$ correspond to rows of M . An entry of M is 1 if the respective element is contained in the respective set, and 0 otherwise. We say that M has the *consecutive ones property* (C1P) if there exists a linear order $\prec$ on S such that for every set $X \in \mathcal{C}$, 1's appear consecutively in the row of M corresponding to X (i.e., if $s_i \prec s_k \prec s_j$ and $s_i, s_j \in X$, then $s_k \in X$). We say that $\mathcal{C}$ has C1P if its corresponding matrix M has C1P. The property C1P, which is originally used for recognition of interval graphs, was first introduced in [8], and restated in an equivalent way in [6]. For the sake of conciseness, we denote the red-blue hitting set problem with $\mathcal{C}_r \cup \mathcal{C}_b$ having the property C1P by *RBHS-C1P*.

1.1 Previous results on the red-blue hitting set problem

Recently, Dom et al. [6] proved that RBHS is **NP-hard** when at most one of $\mathcal{C}_r$ and $\mathcal{C}_b$ has the consecutive ones property. Furthermore, they showed that RBHS-C1P can be solved in $O(|\mathcal{C}_r||\mathcal{C}_b||S|^2)$ time by making use of the dynamic programming strategy. The above results are summarized in Table 1.

RBHS	Hardness
No C1P restriction	NP-hard
$\mathcal{C}_b$ has C1P	NP-hard
$\mathcal{C}_r$ has C1P	NP-hard
$\mathcal{C}_r \cup \mathcal{C}_b$ has C1P	$O(\mathcal{C}_r \mathcal{C}_b S ^2)$

Table 1: Hardness results of RBHS proved by Dom et al. [6].

1.2 Our results

In this paper, by utilizing the consecutive ones property and transforming RBHS-C1P to a shortest path problem in a weighted and acyclic digraph, we devise an $O(|\mathcal{C}_b||S| + |\mathcal{C}_r||S| + |S|^2)$ algorithm for RBHS-C1P. Our algorithm improves the previous time bound $O(|\mathcal{C}_r||\mathcal{C}_b||S|^2)$ proposed by Dom et al. [6]. The rest of this paper is organized as follows. In Sect. 2, some preliminaries for our algorithm will be given. In Sect. 3, we will introduce our algorithm for RBHS-C1P as well as the analysis.

2 Preliminaries

Given the set S and the collection $\mathcal{C} = \mathcal{C}_r \cup \mathcal{C}_b$, the linear order $\prec$ of S which makes $\mathcal{C}$ have C1P can be found in $O(|S| + |\mathcal{C}| + \sum_{X \in \mathcal{C}} |X|)$ time [8]. By considering the above task of finding $\prec$ as a preprocessing step, hereafter, we assume without loss of generality that the elements of the input set S are already sorted according to the order $\prec$ and the collection $\mathcal{C} = \mathcal{C}_b \cup \mathcal{C}_r$ has the consecutive ones property by the order $\prec$. For every $X \in \mathcal{C}_r \cup \mathcal{C}_b$, its *leftmost index*, say, $l(X)$, is defined to be $\min\{i \mid s_i \in X\}$, and its *rightmost index*, say $r(X)$, is defined to be $\max\{i \mid s_i \in X\}$. We assume that the sets in $\mathcal{C}_b$ are sorted to $B_1, \dots, B_{|\mathcal{C}_b|}$ by their leftmost indices in the increasing order according to $\prec$, and the sets in $\mathcal{C}_r$ are also sorted to $R_1, \dots, R_{|\mathcal{C}_r|}$ by their leftmost indices in the increasing order according to $\prec$.

3 Solving the red-blue hitting set problem with the consecutive ones property

In this section, we present an improved polynomial-time algorithm that solves RBHS-C1P in $O(|\mathcal{C}_b||S| + |\mathcal{C}_r||S| + |S|^2)$ time. Recall the problem RBHS-C1P as follows.

The red-blue hitting set problem with the consecutive ones property (RBHS-C1P).

Input: Given a set $S = \{s_1, \dots, s_n\}$ and two collections $\mathcal{C}_b$ and $\mathcal{C}_r$, where $\mathcal{C}_b \cup \mathcal{C}_r$ has C1P according to the order $\prec$ of S .

Task: Find a subset $S' \subseteq S$ such that for every $X \in \mathcal{C}_b$, we have $|S' \cap X| \geq 1$, and $|\{X \in \mathcal{C}_r \mid S' \cap X \neq \emptyset\}|$ is minimized.

By the argument from Dom et al. [6], we may assume that $l(B_i) \neq l(B_j)$, $r(B_i) \neq r(B_j)$, and $l(B_i) \prec l(B_j)$ iff $r(B_i) \prec r(B_j)$ for any $B_i, B_j \in \mathcal{C}_b$. The reason for these assumptions is that whenever a set in $\mathcal{C}_b$ is hit by some element in S , then its supersets in $\mathcal{C}_b$ must be also hit by this element. In fact, to make these assumptions hold, first we scan over $\mathcal{C}_b$ and record each $X \in \mathcal{C}_b$ into a table T_l according to its leftmost element $l(X)$, and another table T_r according to its rightmost element $r(X)$. When two sets X, Y are recorded into the same bucket of T_l , we keep the one, say X without loss of generality, whose rightmost element precedes the other one, and remove Y from $\mathcal{C}_b$. Similarly for T_r ,

we keep the one, say X , whose leftmost element succeeds the other one, and remove Y from $\mathcal{C}_b$. Then for each $X \in \mathcal{C}_b$, we examine each set Y with $l(Y) = p$ for each $l(X) \prec p \preceq r(X)$. Once a set Y with $r(Y) \prec r(X)$ is found, we remove X from $\mathcal{C}_b$. Eventually, the remaining collection $\mathcal{C}_b$ is the one satisfying the assumptions, and the above preprocessing work costs $O(|\mathcal{C}_b||S|)$ time.

3.1 The algorithm

The general concepts of our algorithm are as follows. First, we transform the input instance into an auxiliary weighted and acyclic digraph. Vertices of the graph stand for elements in S . Edges of the graph and weights of edges are carefully assigned. We add two additional vertices as source and sink. Then, we can solve the problem RBHS-C1P by trying to find a shortest path from the source to the sink on this graph.

As what is introduced at the beginning of this subsection, we construct an auxiliary digraph $G = (V, E)$, which is acyclic and weighted. The vertices of G stand for the elements in $S \cup \{s_0, s_{n+1}\}$, where s_0 and s_{n+1} are two additional vertices, which are denoted by *source* and *sink* respectively. With slight abuse of notation, we denote the vertices of G by the elements in $S \cup \{s_0, s_{n+1}\}$. We denote by (x, y) the edge directed from x to y . The edges of G are given in the following way. First, we put the edges (s_0, s_i) for all $s_i \in B_1$, as well as the edges (s_j, s_{n+1}) for all $s_j \in B_{|\mathcal{C}_b|}$. The rest edges are added as follows. For every $s_i \in S$, we find out two sets $B_{k'}, B_{k^*} \in \mathcal{C}_b$, where k' and k^* are the minimum index and maximum index among all sets in $\mathcal{C}_b$ hit by s_i , respectively, and we denote k' and k^* by $h_L(s_i)$ and $h_R(s_i)$ respectively. If an element s_i does not hit any set in $\mathcal{C}_b$, we set $h_L(s_i) = \infty$ and $h_R(s_i) = -\infty$. Then for every two vertices $s_i, s_j \in V$ where $i < j$, we add the edge (s_i, s_j) if $h_L(s_j) - h_R(s_i) \leq 1$. That is, we do not put an edge (s_i, s_j) on G whenever the indices of the sets in $\mathcal{C}_b$ hit by $\{s_i, s_j\}$ do not occur consecutively. In addition, there is no cycle on G since there is no edge (s_j, s_i) where $s_j \succ s_i$.

Consider Fig. 2 as an example. Assume that we are given a set of elements $S = \{s_1, s_2, s_3, s_4, s_5, s_6, s_7\}$, and a blue collection $\mathcal{C}_b = \{B_1, B_2, B_3\}$, where $B_1 = \{s_1, s_2, s_3\}$, $B_2 = \{s_4, s_5\}$, and $B_3 = \{s_5, s_6, s_7\}$ (see Fig. 1). The corresponding auxiliary digraph by this instance is shown in Fig. 2. From Fig. 1 we can obtain that $h_L(s_3) = h_R(s_3) = 1$, $h_L(s_5) = 2$, $h_R(s_5) = 3$, $h_L(s_6) = h_R(s_6) = 3$. Then we put an edge

(s_3, s_5) since $h_L(s_5) - h_R(s_3) = 1$, yet we do not put the edge (s_3, s_6) on the graph since $h_L(s_6) - h_R(s_3) > 1$.

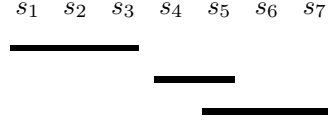


Fig. 1: Illustration of the $\mathcal{C}_b$ and the set S of an instance of RBHS-C1P.

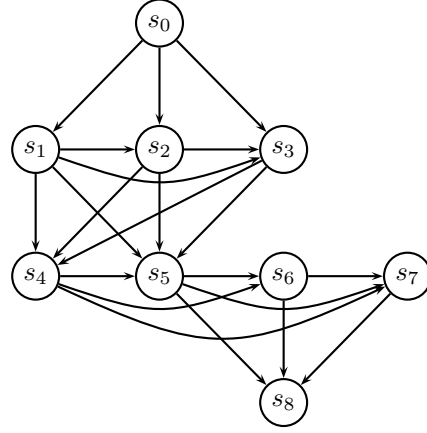


Fig. 2: An auxiliary digraph constructed by the example given in Sect. 3.1.

Lemma 1. *If a subset $S' \subseteq S$ that hits all sets in $\mathcal{C}_b$, then $S' \cup \{s_0, s_{n+1}\}$ forms a path from s_0 to s_{n+1} on the auxiliary digraph G .*

Proof. Let $S' = \{s_{i_1}, s_{i_2}, \dots, s_{i_{|\mathcal{C}_b|}}\}$ denotes a subset of S such that s_{i_j} is one of the elements hitting B_j , for $1 \leq j \leq |\mathcal{C}_b|$, and $s_{i_1} \preceq s_{i_2} \preceq \dots \preceq s_{i_{|\mathcal{C}_b|}}$. Since $h_R(s_{i_j}) \geq j$, $h_L(s_{i_{j+1}}) \leq j+1$, we have $h_L(s_{i_{j+1}}) - h_R(s_{i_j}) \leq 1$, for all $1 \leq j \leq |\mathcal{C}_b| - 1$. Thus, if $s_{i_j} \neq s_{i_{j+1}}$, $(s_{i_j}, s_{i_{j+1}}) \in E$ for all $1 \leq j \leq |\mathcal{C}_b| - 1$. Since $(s_0, s_{i_1}), (s_{i_{|\mathcal{C}_b|}}, s_{n+1}) \in E$, we obtain that $s_0 \rightarrow s_{i_1} \rightarrow \dots \rightarrow s_{i_{|\mathcal{C}_b|}} \rightarrow s_{n+1}$ is a path from s_0 to s_{n+1} . Hence the lemma is proved. $\square$

Lemma 2. *If P is a path from s_0 to s_{n+1} on the auxiliary digraph G , then S' hits all sets in $\mathcal{C}_b$, where $S' \subseteq S$ are the set of vertices on P except s_0 and s_{n+1} .*

Proof. Note that if s_i does not hit any set in $\mathcal{C}_b$, then for any $s_p \in S \setminus \{s_i\}$, there is neither an edge

(s_p, s_i) nor an edge (s_i, s_p) on G since $h_L(s_i) = \infty$ and $h_R(s_i) = -\infty$. So we can assume that each vertex s_i on the path corresponds to an element which must hit some set(s) in $\mathcal{C}_b$. We prove this lemma by contradiction as follows. Then suppose that there exists a path $P : s_0 \rightarrow \dots \rightarrow s_{i'} \rightarrow s_{j'} \rightarrow \dots \rightarrow s_{n+1}$ such that $s_{i'} \in B_{i'}$ and $s_{j'} \in B_{j'}$, where $j' - i' > 1$, and some $B_k \in \mathcal{C}_b$, $i' < k < j'$, is not hit by the elements corresponding to the vertices on P . Then $h_L(s_{j'}) - h_R(s_{i'}) \geq (k+1) - (k-1) > 1$, we have $(s_{i'}, s_{j'}) \notin E$ which contradicts to the assumption that P is a path. Therefore the lemma is proved. $\square$

By Lemma 1 and 2, we immediately have the following theorem.

Theorem 1. *A subset $S' \subseteq S$ hits all sets in $\mathcal{C}_b$ if and only if $S' \cup \{s_0, s_{n+1}\}$ forms a path from s_0 to s_{n+1} on the auxiliary digraph G .*

After the auxiliary digraph G is constructed, we introduce how to assign weights on edges of G as follows. Let $w(s_i)$ be the number of sets in $\mathcal{C}_r$ hit by s_i . The *edge weight* of (s_i, s_j) , denoted by $w(s_i, s_j)$, is defined to be $w(s_j) - w'(s_i, s_j)$, where $w'(s_i, s_j) = |\{X \in \mathcal{C}_r : s_i, s_j \in X\}|$ is called the *overcounted weight* of (s_i, s_j) . Actually, the edge weight $w(s_i, s_j)$ is the number of the sets in $\mathcal{C}_r$ which are hit by s_j but not by s_i . Note that $w'(s_0, s_j) = 0$ (i.e., $w(s_0, s_j) = w(s_j)$) for $s_j \in B_1$, and $w'(s_i, s_{n+1}) = 0$ (i.e., $w(s_i, s_{n+1}) = 0$) for $s_i \in B_{|\mathcal{C}_b|}$.

Algorithm 1 : An improved algorithm for RBHS-C1P.

- 1: Construct an auxiliary weighted acyclic digraph $G = (V, E)$.
 - 2: Compute $w(s_i) = |\{R_k \in \mathcal{C}_r \mid s_i \in R_k\}|$ for each vertex $s_i \in S$ ($w(s_0) = w(s_{n+1}) = 0$).
 - 3: For each $s_i \in S$, record $\{R_k \in \mathcal{C}_r \mid s_i \in R_k\}$ into a table according to their leftmost elements.
 - 4: Perform the initialization $w'(s_0, s_j) = 0$ for all $s_j \in S$, and then iteratively compute the overcounted weight $w'(s_i, s_j) = w'(s_{i-1}, s_j) + |\{R_k : l(R_k) = s_i, r(R_k) \succeq s_j\}|$, for all $i < j$.
 - 5: Assign the edge weight $w(s_i, s_j) = w(s_j) - w'(s_i, s_j)$ for each edge (s_i, s_j) .
 - 6: Find out the shortest path on G from s_0 to s_{n+1} and collect the elements on the path to S' excluding s_0 and s_{n+1} .
-

After the edge weights are assigned, we use the Dijkstra's algorithm [4, 10] to find the shortest

path on G . Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted and acyclic digraph $G = (V, E)$ for the case in which all edge weights are nonnegative. The weight of each edge (s_i, s_j) is assigned by $w(s_i, s_j) = w(s_j) - w'(s_i, s_j)$. First, we assign every vertex a distance value. Set the source vertex s_0 to zero and other nodes to infinity. Initially, assume that all vertices are unvisited and set the source vertex to be the current node. We consider all unvisited neighbors v of the current node u and compute their distance between u and v by their edge weights. We can get the shortest path from the source (i.e., s_0) to the sink (i.e., s_{n+1}) by Dijkstra's algorithm and then collect the vertices on the shortest path except s_0 and s_{n+1} into the set S' , then we have the solution to the problem RBHS-C1P.

Algorithm 1 summarizes the above steps of our algorithm for solving the problem RBHS-C1P.

Theorem 2. *Algorithm 1 solves the problem RBHS-C1P.*

Proof. By Theorem 1 we know that Algorithm 1 satisfies the “blue” restriction. To prove that Algorithm 1 ensures the “red” restriction, we first prove that the sum of edges weights on the path $P = s_0 \rightarrow s_{r_1} \rightarrow \dots \rightarrow s_{r_t} \rightarrow s_{n+1}$ is equal to the number of sets in $\mathcal{C}_r$ hit by $\{s_{r_1}, s_{r_2}, \dots, s_{r_t}\}$.

Note that for $r_p < r_i < r_j$, any set hit by s_{r_p} and s_{r_j} must be hit by s_{r_i} due to the consecutive ones property. Thus when an edge (s_{r_i}, s_{r_j}) is added to the path $s_0 \rightarrow s_{r_1} \rightarrow \dots \rightarrow s_{r_{i'}} \rightarrow s_{r_i}$, the collection of sets in $\mathcal{C}_b$ hit by s_{r_i} and s_{r_j} includes those hit by s_{r_p} and s_{r_j} , for each $s_{r_p} \in \{s_{r_1}, \dots, s_{r_{i'}}, s_{r_i}\}$. Since $w(s_{r_j})$ is the number of sets in $\mathcal{C}_r$ hit by s_{r_j} , we have that $w(s_{r_i}, s_{r_j})$, i.e., $w(s_{r_j}) - w'(s_{r_i}, s_{r_j})$, is the number of sets in $\mathcal{C}_r$ hit by s_{r_j} but not by $\{s_{r_1}, \dots, s_{r_{i'}}, s_{r_i}\}$. Thus by summing up the edge weights on the path P , we can obtain the number of sets in $\mathcal{C}_r$ hit by $\{s_{r_1}, s_{r_2}, \dots, s_{r_t}\}$, which are the vertices on P except s_0 and s_{n+1} .

Since a shortest path on G from s_0 to s_{n+1} is the path from s_0 to s_{n+1} with the smallest sum of edge weights, using Dijkstra's algorithm for finding a shortest path from s_0 to s_{n+1} ensures the “red” restriction. By the above arguments, therefore, Algorithm 1 solves RBHS-C1P. $\square$

3.2 Time complexity of the algorithm

We give a sketch of the analysis of this subsection here. First, we show that constructing the auxiliary digraph G can be done in $O(|\mathcal{C}_b||S| + |S|^2)$

time. Second, we show that it takes $O(|\mathcal{C}_r||S| + |S|^2)$ time to compute the weights of edges in G . Finally, we finish our analyses by a concluding theorem.

Lemma 3. *The auxiliary weighted acyclic digraph $G = (V, E)$ can be constructed in $O(|\mathcal{C}_b||S| + |S|^2)$ time.*

Proof. First we make a vertex for each element in S . For each vertex s_i , we find out $h_L(s_i)$ and $h_R(s_i)$ and record them into a table, and it takes $O(|\mathcal{C}_b|)$ time to do these jobs since checking whether a set, say B_k , is hit by s_i can be done in constant time by simply checking if $l(B_k) \preceq s_i \preceq r(B_k)$. Then for every two vertices s_i and s_j , where $i < j$, we check if $h_L(s_j) - h_R(s_i) \leq 1$ and add an edge (s_i, s_j) if this inequality holds. Such checking and an edge addition cost $O(1)$ time for every two vertices s_i and s_j . Therefore by the above arguments, G can be constructed in $O(|\mathcal{C}_b||S| + |S|^2)$ time. $\square$

Lemma 4. *All the overcounted weights (i.e., $\{w'(s_i, s_j) \mid i < j\}$) can be computed in $O(|\mathcal{C}_r||S| + |S|^2)$ time.*

Proof. We utilize some tricks for computing the overcounted weights as follows. For each $s_i \in S$, we search in $\mathcal{C}_r$ for the sets hit by s_i , and then record them into a table according to their leftmost elements. We also keep the number of sets recorded in each bucket. For all $s_j \in S$, we initialize $w'(s_0, s_j)$ to be 0. Then, due to the consecutive ones property, for all $i < j$ we can compute $w'(s_i, s_j)$ by the formula $w'(s_i, s_j) = w'(s_{i-1}, s_j) + |\{R_k : l(R_k) = s_i, r(R_k) \succeq s_j\}|$, where the last term stands for the number of sets in $\mathcal{C}_r$ hit by s_i and s_j but not by s_{i-1} . Afterwards, we can make use of the table and the above formula to compute every overcounted weight in an iterative way.

The initialization of setting $w'(s_0, s_j) = 0$ for all $s_j \in S$ costs $O(|S|)$ time. It costs $O(|\mathcal{C}_r|)$ time to search for the sets in $\mathcal{C}_r$ hit by an element s_i and record them according to their leftmost elements into a table. Hence this process overall takes $O(|\mathcal{C}_r||S|)$ time. Then, when we count the overcounted weight $w'(s_i, s_j)$ by $w'(s_i, s_j) = w'(s_{i-1}, s_j) + |\{R_k : l(R_k) = s_i, r(R_k) \succeq s_j\}|$, we only need to count the number of sets $R_k \in \mathcal{C}_r$ hit by s_j such that $l(R_k) = s_i$. Each of these countings can be done by making use of the table in $O(1)$ time, so all the overcounted weights can be computed iteratively in $O(|S|^2)$ time. Therefore, all of the above processes for computing the

overcounted weights cost at most $O(|\mathcal{C}_r||S| + |S|^2)$ time. $\square$

After all the overcounted weights are computed, we can assign each edge (s_i, s_j) of G an edge weight $w(s_i, s_j)$ in constant time, so all the edge weights can be assigned in at most $O(|S|^2)$ time. As for finding the shortest path on G , we apply Dijkstra's $O(|S|^2)$ algorithm [4]. Therefore, we have the following concluding theorem.

Theorem 3. *The red-blue hitting set problem with the consecutive ones property can be solved in $O(|\mathcal{C}_b||S| + |S|^2 + |\mathcal{C}_r||S|)$ time.*

References

- [1] D. Brinza, A. Zelikovsky: Design and Validation of Methods Searching for Risk Factors in Genotype Case-Control Studies. *Journal of Computational Biology* **15**(1) (2008) 81–90.
- [2] R. D. Carr, S. Doddi, G. Konjevod, and M. V. Marathe: On the red-blue set cover problem. In *Proceedings of the 11th annual ACM-SIAM symposium on Discrete algorithms (SODA'00)*, San Francisco, California, pp. 345–353.
- [3] A. Caprara, G. F. Italiano, G. Mohan, A. Panconesi, and A. Srinivasan: Wavelength rerouting in optical networks, or the Venetian routing problem. *J. Algorithms* **45** (2002) 93–125.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein: *Introduction to Algorithms, Second Edition*. MIT Press and McGraw-Hill, 2001.
- [5] M. Dom, J. Guo, R. Niedermeier, and S. Wernicke: Minimum Membership Set Covering and the Consecutive Ones Property. *Proc. 10th SWAT-06, Vol. 4059 in LNCS* (2006) 339–350.
- [6] M. Dom, J. Guo, R. Niedermeier, and S. Wernicke: Red-blue covering problems and the consecutive ones property. *J. Discrete Algorithms* **6** (2008) 393–407.
- [7] U. Feige: A threshold of $\ln n$ for approximating set cover. *J. ACM* **45** (1998) 634–652.
- [8] D. R. Fulkerson and O. A. Gross: incidence matrices and interval graphs. *Pacific J. Math.* **15** (1965) 835–855.

- [9] David Peleg: Approximation algorithms for the Label-CoverMAX and Red-Blue Set Cover problems . *The Weizmann Institute of Science, Rehovot 76100, Israel*. (2006).
- [10] F. B. Zhan, and C. E. Noon: Shortest path algorithms: an evaluation using real road networks. *Transportation Science* **32** (1998) 65-73.